

Insight Analyst – AI-Powered Log Summarizer

Martin Quintana

Indiana Institute of Technology

CYS 4200: Cybersecurity Project II



December 5, 2025

Insight Analyst – AI-Powered Log Summarizer

Artificial intelligence (AI) is the hottest topic across the entire globe right now and for good reason. From developing applications to becoming an academic weapon, AI has been implemented into multiple industries. Its addition into the cybersecurity realm has been a slower process than some may have imagined, but it seems to have finally reached a near-autonomous read team ability. This means the agent is able to act on its own as a threat actor, without much human intervention. This coupled with rapid advancement in the field can lead to a very grim future. Insight Analyst is meant to serve as an equation that can help provide an answer for this problem.

Scenario

Gathering and analyzing logs takes a long time, especially when done properly alongside note taking. Allowing a first-responder SOC analyst to skip these steps can be the difference between saving a company and hundreds of thousands of people having their personally identifiable information (PII) leaked. Insight Analyst will take this burden off analysts so that their main focus can shift towards securing the environment. Faster triage will directly result in a faster time to resolution and overall system security. Realistically, this tool can be used by any cybersecurity professional but was designed with SOC analysts as the target audience.

Model Constituents

Hardware

The hardware for this project is arguably more important than the software, as it allows the usage of certain software, or in this case, the ability to run certain models. None of this hardware was chosen with running a local LLM instance in mind. The system includes the following: Intel Core i9-12900K 16-core processor, MSI MAG Z790 Tomahawk DDR4

motherboard, Corsair Pro 32 GB DDR4 CL18 RAM at 3600 MHz, Samsung 990 Pro PCIE 4.0 x 4 M.2 NVME, and a Gigabyte OC GeForce RTX 4070 GPU with 12 GB of VRAM. This is all an adequate amount to run a model that is both quick and trained on enough material to produce quality responses. The most important item is the GPU; who's VRAM determines the size of the model that is able to run. A 12 GB GPU can handle many different parameter models, such as 9 to 13 billion or even 30 billion Mixture-of-Experts (MoE) models. The CPU is another main component as it can contribute massively to LLM performance on a system. It helps with speeding up the model's logic, it determines the bandwidth for RAM, and it assists with managing GPU tasks. A large-enough amount of RAM is also crucial as it can affect token generation, the length of context windows, and the number of requests that can be handled. Storage also plays a part, especially when it comes to GPU data loading. Faster storage will allow the GPU to access this data at a quicker pace, resulting in a faster product. All of these items are reliant on the motherboard, the backbone of the host. Ensuring compatibility with parts and PCIe speeds will ensure smooth operations and each item being utilized to its full ability.

The Selection Process

The array of hardware will have a direct impact on the model that is chosen. Large models that can run on 12 GB of VRAM are favorable, but not necessary. This project's main purpose is to produce summaries and provide a small slice of reasoning, hence a relatively small-sized 7B parameter model. A larger model will slow down the summary process and consume more system resources. Moving on from parameters, another necessary consideration is the style of LLM. There are two different options to consider: instruction and domain-specific models. Both are great in their own respects, but an instruction model makes more sense here as it is mainly summarizing. If the model needed to reason a bit more and include more complex threat

hunting methodologies, then a domain-specific model would be the better option. With these criteria, there are a few different models that can be considered. Mistral 7B, Llama 3.1 8B, and Qwen2.5 7B were all the main candidates for the model. Qwen2.5 was ultimately chosen as it tends to provide technical and factually accurate responses. It is also a middle ground for Llama's reasoning capability and Mistral's processing speed. Qwen2.5 is proven to be a very efficient model when compared to others, especially in its class of smaller instruct models.

Tools & Technologies

Platform

LM Studio serves as the platform to run the Qwen2.5 model. It is the glue that allows the web app to interact with the LLM. This provides an easy integration into Windows with a simple interface. Using this application avoids the need for something like PyTorch, a Python framework that is used for training LLMs. Including PyTorch would complicate his project and possibly cause more incompatibility issues. This is all running on a Windows 11 operating system, so compatibility is an important factor.

Following this, the model must have an environment to run inside of. That's where virtual environments in PowerShell show up. It is an isolated system that allows the user to run whatever they desire. This includes but is not limited to installing packages, running different operating systems, the safe development of tools without other factors that may affect them, and having a replica of the current system for test-case purposes. Insight Analyst took advantage of this feature when downloading packages. This will result in a quicker cleanup process for removing or upgrading packages, efficient use of resources, and no conflicting libraries that will cause bugs. To launch the application, a simple PowerShell script must be run. The script will change directories to the project folder and start the virtual environment (see Appendix A).

Example Logs

All example CSVs were gathered from a Windows Server 2022 host that had Mimikatz.exe ran on it. Logs were then exported using Splunk Universal Forwarder and Windows Event Viewer. The Splunk Indexer, the log collector, was a Debian 12 host that had Splunk Server installed on it. Both hosts were on the same network. The example logs were produced for the sake of testing, and no testing was done outside of a consenting practice environment with resources provided by the Indiana Tech Cyber Warriors.

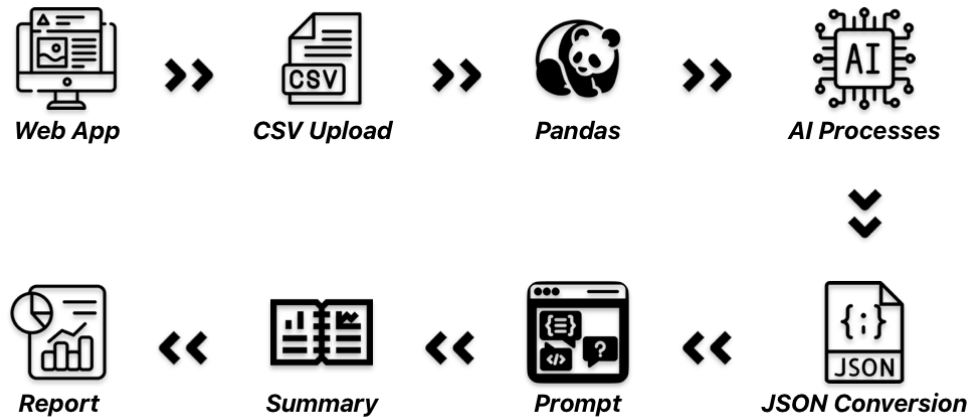
Python

All of the libraries and tooling that made this project possible were Python-based. The entire web app is even written in Python. This makes front and back-end development very simple as it is only one file and one programming language. With the addition of libraries, there is no need for extra files or languages. To start off, Python 3.12 was used to ensure a stable version of Windows wheels. Wheels from Python allows for the quick installation of packages. It essentially skips the step of zipping and unzipping applications. It even allows for operating system-specific distributions, ensuring compatible software. Some of the main libraries used were Streamlit, Pandas, dateutil, and openai. Streamlit is a web framework that focuses on UI and user interaction. It will turn the Python code into a web app. It also has built-in file uploading, ability to download a file, and it displays the preview of the CSV table. Pandas is used for processing the data from the CSV. It will fix formatting, extract fields, build the preview table for Streamlit, and it determines how the data is sent to the model. Dateutil is responsible for timestamp formatting. Openai initializes the client that talks to LM Studio's local API Server. It serves as the bridge between the model and all of the Python code. Python's importance cannot be understated as it is the layer allowing for this project to blossom.

Project Architecture

Figure 1

General Architecture Flow Diagram



The architecture of Insight Analyst follows a specific path, ensuring the LLM takes certain steps before producing the final result. The overarching steps are provided in Figure 1.

Explanations for the precise steps are as follows:

1. The end user will launch the web app using Streamlit UI. Streamlit is a Python library that will load the app.py file containing the front and back-end code.
2. The user will then upload a CSV file for analyzing. This can be from Splunk or Windows Event Viewer.
3. Pandas will read the CSV and ensure a smooth hand-off of data to the app. Pandas is a Python library used for reading data from files, it enables sorting, and it allows developers to combine different datasets. Alongside this is a function to help align Windows Event Viewer columns. This ensures correct data display in the Preview section of the final result.
4. The next step is to process this data; however, this may be an issue with larger files. To prevent any errors, the data must be truncated. This data will be sent to the model in

multiple chunks, ensuring there are no overflows. This includes row limiting and only extracting a limited number of characters. The row count is limited to 200 and the characters are capped at 300.

5. This data will now need to be converted to JSON. Models do not understand CSV files, but rather structured data that is labeled. Converting to JSON is one of the best ways to prompt any model, as it will easily follow instructions, without having to clarify more than once.
6. All extracted data is now ready to send off, alongside the pre-defined prompt in the code. This prompt is sent by calling the local API server on the system. The model will respond as if it were being prompted in a chat.
7. A summary of findings will be generated, including a preview of the CSV contents, an incident summary, Event IDs gathered, the scenario that unfolded, possible MITRE ATT&CK techniques, related Event IDs to look into, applications or services that should be looked into, how to contain and remove the malware or threat actor, future steps to ensure a hardened system, guidelines on verifying a clean machine, and a full markdown report.
8. All of this will now be displayed to the screen. Alongside that is the ability to download the markdown report that was created. The user can open this report in multiple applications, no need to worry about compatibility or needing a specific application to read the contents. This report is a copy of what was displayed to the screen.

Next Steps

While this project has fulfilled some ideas and the main goal, there is room for improvement. The first thing that needs to be worked on is the ability to upload multiple CSVs

for analysis. The model can currently only handle a single file, for a reason that has not been investigated. Next will be to create a list or dashboard of previous submissions. This will allow users to go back on previous uploads in case it relates to a new incident. Lastly, building this project on Linux is the ultimate goal. This will allow for more resource usage, so a larger model that can summarize and provide more reasoned opinion. A new model must also have more knowledge in the realm of cybersecurity, hence a larger size model. Building this on Linux will also allow for much easier package management since there will be no Windows dependencies.

Conclusion

Insight Analyst has proven to be an invaluable project as learning material. From researching different AI topics to prompting a model in different ways, there are plenty of takeaways. High-end hardware, compatible tools and software, educated selection on model options, designing the usage and the final result have all contributed to the success of this project. However, the most important takeaway is that technology will never stop progressing. Taking on every challenge as a learning opportunity is a guaranteed path to success. Whether in life or work, there is always something worth reaching for.

References

<https://lmstudio.ai/docs/developer/core/headless>

<https://lmstudio.ai/docs/developer/rest/endpoints>

<https://github.com/lmstudio-ai/lms/issues/196>

<https://www.dhiwise.com/post/text-summarization-models-that-support-better-focus>

<https://huggingface.co/Qwen/Qwen2.5-7B-Instruct>

https://dataloop.ai/library/model/qwen_qwen25-15b-instruct/#model-overview

<https://scrapingant.com/blog/llm-instruct-vs-chat>

<https://artificialanalysis.ai/models/qwen2.5-32b-instruct>

<https://streamlit.io>

<https://pandas.pydata.org/docs/>

<https://docs.python.org/3/library/venv.html>

<https://mpgone.com/json-prompt-guide/>

<https://github.com/ParrotSec/mimikatz>

https://www.splunk.com/en_us/download/splunk-enterprise.html

Appendix A

PowerShell App-Launching Script

```
cd C:\LogSummarizer
```

```
.\.venv\Scripts\Activate.ps1
```

```
python -m streamlit run app.py
```

Appendix B

Python Application

```

import json
import pandas as pd
import streamlit as st
from openai import OpenAI

# LM Studio / model config
LM_BASE_URL = "http://localhost:1234/v1" # LM Studio server
LM_MODEL = "mistral-7b-instruct" # model
API_KEY = "lm-studio"

TEMP = 0.2
MAX_TOKENS = 550
MAX_ROWS_TO_MODEL = 200 # cap rows sent
MAX_CHARS_PER_FIELD = 300 # cap characters sent

# Streamlit page setup
st.set_page_config(
    page_title="Insight Analyst — Log Summarizer",
    layout="wide"
)

st.title("Insight Analyst — Log Summarizer")
st.caption(
    "Upload a CSV from Windows Event Viewer or Splunk."
    " The app shows a small preview, generates an incident-style AI summary, "
    "and suggests further investigation steps using a local model."
)

# UI fixes
st.markdown(
    """
<style>
/* Keep content nicely centered with max width */
.block-container {
    max-width: 1200px;
    margin: auto;
}

/* Center headers and cells in tables/dataframes */
table, th, td { text-align: center !important; }
[data-testid="stDataFrame"] td, [data-testid="stDataFrame"] th {
    text-align: center !important;
    """

```

```

}

/* Hide toolbar buttons (fullscreen/search/download) on dataframes */
[data-testid="stDataFrame"] [data-testid="stElementToolbar"] {
  display: none !important;
}

/* Hide column menu (hide/pin/etc.) */
[data-testid="stDataFrame"] [role="button"] {
  display: none !important;
}
</style>
"""
    ,
    unsafe_allow_html=True,
)

# fix Windows Event Viewer logs
def fix_windows_event_viewer_csv(df: pd.DataFrame) -> pd.DataFrame:
    """
    Windows Security CSV (like the one you uploaded) has:
    headers: Keywords,Date and Time,Source,Event ID,Task Category
    rows: Audit Success,11/13...,Microsoft...,4624,Logon,<Message>

    That means there are 6 values for 5 headers, and pandas shifts everything
    left and uses 'Audit Success' as the index. We fix that here and produce:

    Keywords | Date and Time | Source | Event ID | Task Category | Message
    """
    expected_cols = ["Keywords", "Date and Time", "Source", "Event ID", "Task Category"]

    if list(df.columns) == expected_cols and df.index.dtype == object:
        # Reconstruct the correct columns using index + shifted columns
        fixed = pd.DataFrame({
            "Keywords": df.index,
            "Date and Time": df["Keywords"].values,
            "Source": df["Date and Time"].values,
            "Event ID": df["Source"].values,
            "Task Category": df["Event ID"].values,
            "Message": df["Task Category"].values,
        })
        fixed.reset_index(drop=True, inplace=True)
        return fixed

    # Otherwise, leave as-is
    return df.reset_index(drop=True)

# extracting time content

```

```

def add_time_for_ai(df: pd.DataFrame) -> pd.DataFrame:
    """
    For the AI-only copy: if there's a time-like column, copy it to _time
    WITHOUT renaming or removing the original column.
    """
    if "_time" in df.columns:
        return df

    candidates = [
        "Date and Time",
        "TimeCreated",
        "Time Generated",
        "Timestamp",
        "Time",
        "Created",
        "EventTime",
        "@timestamp",
    ]
    lower_map = {c.lower(): c for c in df.columns}
    for cand in candidates:
        if cand.lower() in lower_map:
            df["_time"] = df[lower_map[cand.lower()]]
            return df

    df["_time"] = None
    return df

# need to list columns that AI should look at
def pick_useful_columns(df: pd.DataFrame) -> pd.DataFrame:
    """Keep only fields that are usually interesting for incident summaries."""
    useful_names = {
        "_time",
        "time",
        "timestamp",
        "eventcode",
        "event id",
        "event_id",
        "computername",
        "machine",
        "hostname",
        "targetusername",
        "subjectusername",
        "user",
        "account name",
        "ipaddress",
        "source_network_address",
        "ip",
    }

```

```

    "image",
    "process",
    "commandline",
    "command line",
    "message",
    "task category",
    "keywords",
    "source",
    "provider name",
    "logon type",
    "date and time",
}
keep = []
for c in df.columns:
    if str(c).lower() in useful_names:
        keep.append(c)
return df[keep] if keep else df

# preview table
def build_preview_table(df: pd.DataFrame) -> pd.DataFrame:
    """
    Preview:
    If we have standard fixed Windows Security columns, show exactly:
        Source | Event ID | Task Category | Date and Time
    otherwise show the first few raw columns.
    """
    cols = df.columns
    if all(c in cols for c in ["Source", "Event ID", "Task Category", "Date and Time"]):
        return df[["Source", "Event ID", "Task Category", "Date and Time"]].head(25).copy()

    return df.head(25).copy()

# send data off in chunks that can be accepted
def compress_df_for_model(df_small: pd.DataFrame) -> pd.DataFrame:
    """
    Reduce the size of the data we send to the model:
    - Limit number of rows (sample evenly across the file)
    - Truncate very long text fields
    """
    df = df_small.copy().astype(str)

    # limit rows
    if len(df) > MAX_ROWS_TO_MODEL:
        step = max(1, len(df) // MAX_ROWS_TO_MODEL)
        df = df.iloc[::step].head(MAX_ROWS_TO_MODEL)

```

```

# shorten long text in each column
for c in df.columns:
    df[c] = df[c].str.slice(0, MAX_CHARS_PER_FIELD)

return df

# local LLM (LM Studio)
client = OpenAI(base_url=LM_BASE_URL, api_key=API_KEY)

# main prompt for output
def ai_global_summary(df_small: pd.DataFrame) -> str:
    """
    Summarize the uploaded log into a single incident-style narrative.
    """
    sample = compress_df_for_model(df_small).to_dict(orient="records")
    messages = [
        {
            "role": "system",
            "content": (
                "You are a professional SOC analyst with over 20 years of experience. You will be given Windows/Splunk\n"
                "CSV logs.\n"
                "Write a concise incident-style summary with 6–10 sentences that includes:\n"
                "- Time window of activity\n"
                "- Main users, hosts, and IPs involved\n"
                "- Notable events and Event IDs (e.g., 4625, 4624, Sysmon IDs)\n"
                "- Likely scenario (e.g., normal sign-in, brute force attempt, suspicious admin action)\n"
                "- MITRE ATT&CK techniques (name and ID) when clearly applicable\n"
                "- Overall risk level and a conclusion.\n"
                "Use a neutral, professional tone and refer to concrete indicators from the data."
            ),
        },
        {
            "role": "user",
            "content": json.dumps(sample, ensure_ascii=False),
        },
    ]
    resp = client.chat.completions.create(
        model=LM_MODEL,
        messages=messages,
        temperature=TEMP,
        max_tokens=MAX_TOKENS,
    )
    return resp.choices[0].message.content

# further investigation (different section) prompt

```

```

def ai_further_investigation(df_small: pd.DataFrame) -> str:
    """
    Suggest follow-up investigation steps and hardening actions based on the data.
    """
    sample = compress_df_for_model(df_small).to_dict(orient="records")
    messages = [
        {
            "role": "system",
            "content": (
                "You are advising a blue-team responder. Based on these log rows, output a practical checklist "
                "with clear format using Markdown:\n\n"
                "Create a checklist header titled Further Investigation:\n\n"
                "## Related Event IDs\n"
                "- List specific Windows / Sysmon Event IDs that should be checked in more detail.\n\n"
                "## Potentially Related Sources\n"
                "- Suggest log sources to correlate (e.g., AD, Defender, firewall, proxy, DNS, EDR).\n\n"
                "## Containment & Removal\n"
                "- If this activity could be malicious, list concrete containment/eradication steps.\n\n"
                "## Hardening & Future Prevention\n"
                "- Recommend configuration changes (GPO, logging, Defender, PS logging, MFA, etc.) to reduce risk.\n\n"
                "## Verify System Integrity\n"
                "- List what should be verified after fixes (e.g., no new admin accounts, no unusual logons).\n\n"
                "Be specific, short, and practical. Reference concrete Event IDs or fields when possible."
            ),
        },
        {
            "role": "user",
            "content": json.dumps(sample, ensure_ascii=False),
        },
    ]
    resp = client.chat.completions.create(
        model=LM_MODEL,
        messages=messages,
        temperature=TEMP,
        max_tokens=MAX_TOKENS,
    )
    return resp.choices[0].message.content

# file upload
uploaded = st.file_uploader(
    "Upload a CSV from Windows Event Viewer or Splunk",
    type=["csv"],
)

if not uploaded:
    st.info("Upload a CSV file to begin.")
else:

```

```

try:
    # read CSV file
    df_raw = pd.read_csv(uploaded, low_memory=False)

    # Windows Event Viewer errors when displaying columns
    df_fixed = fix_windows_event_viewer_csv(df_raw)

    st.success(f"Loaded {len(df_fixed)} rows from {uploaded.name}")

    # Preview section
    st.header("Preview")
    preview = build_preview_table(df_fixed)
    st.dataframe(
        preview,
        use_container_width=True,
        hide_index=True,
    )

    # view for AI
    df_ai = df_fixed.copy()
    df_ai = add_time_for_ai(df_ai)
    df_ai_small = pick_useful_columns(df_ai)

    st.divider()

    # File-Level AI Summary
    st.header("File-Level AI Summary")
    with st.spinner("Asking local model for an incident-style summary..."):
        try:
            summary_text = ai_global_summary(df_ai_small)
            st.markdown(summary_text)
        except Exception as e:
            st.error(f"AI summary failed: {e}")
            summary_text = ""

    # Further Investigation
    st.markdown("---")
    with st.spinner("Asking local model for follow-up recommendations..."):
        try:
            further_text = ai_further_investigation(df_ai_small)
            st.markdown(further_text)
        except Exception as e:
            st.error(f"Further Investigation failed: {e}")
            further_text = ""

    # Download Markdown File
    if summary_text or further_text:
        combined_md = f"""\# Incident Summary

```

```
{summary_text}

---

# Further Investigation

{further_text}

---

*Generated by Insight Analyst*
"""
    st.markdown("## Report Download")
    st.download_button(
        "Download the Full Markdown Report",
        data=combined_md.encode(),
        file_name="insight_analyst_report.md",
        mime="text/markdown",
    )

except Exception as e:
    st.error(f"Failed to read CSV: {e}")
```